

Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ
Обособленное подразделение «Институт информационных технологий
БГУИР»

Факультет повышения квалификации и переподготовки
Кафедра микропроцессорных систем и сетей

ОТЧЕТ

по практическому занятию № 1
по дисциплине «Управление проектами»
на тему «Разработка концепции проекта»

Выполнила: слушатель гр. 60121

К. А. Хаджинова

Проверила:

И. В. Кашникова

Минск 2026

СОДЕРЖАНИЕ

1 ЦЕЛЬ РАБОТЫ	3
2 ХОД РАБОТЫ	4
2.1 Теоретические сведения	4
2.2 Объект проектирования.....	4
2.3 Обоснование проекта по модели 5W	4
2.4 Бизнес-окружение проекта.....	5
2.5 SMART-цель проекта	6
2.6 Правила приёмки и допущения	7
2.7 Содержание и границы проекта	7
2.8 Ограничения проекта.....	9
2.9 Декомпозиция цели проекта	10
2.10 Анализ заинтересованных сторон	12
ВЫВОДЫ.....	14
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	15

1 ЦЕЛЬ РАБОТЫ

Освоение методики разработки концепции программного проекта: обоснование проекта по модели 5W, формулирование SMART-цели, определение критериев приёмки, допущений, содержания и границ проекта с приоритизацией требований по методу MoSCoW, ограничений, декомпозиции цели и анализа заинтересованных сторон на примере проекта NotaCode.

2 ХОД РАБОТЫ

2.1 Теоретические сведения

Концепция проекта разрабатывается на основе анализа потребностей бизнеса и определяет, что и зачем делается в проекте; документ служит для принятия решений в ходе проекта и при приёме результата [1]. Стандартное содержание концепции: название, сроки, цели и результаты проекта, допущения и ограничения, ключевые участники и заинтересованные стороны, ресурсы, риски, критерии приёма и обоснование полезности.

Цель проекта формулируется по критериям SMART: конкретная (Specific), измеримая (Measurable), достижимая (Achievable), значимая (Relevant) и ограниченная во времени (Time-bound) [2]. Требования приоритизируются по методу MoSCoW: Must have, Should have, Could have, Won't have; доля Must have не должна превышать 60 % общего списка. Заинтересованные стороны распределяются по матрице «влияние × интерес» [3].

2.2 Объект проектирования

Объект проектирования – проект NotaCode: веб-IDE класса «diagram as code», в которой диаграмма описывается кодом на собственном предметно-ориентированном языке, проверяется по правилам выбранной нотации и отображается на холсте. Исходные данные о проекте взяты из проектной документации [4]; проект является перезапуском прототипа DiagramCode.

2.3 Обоснование проекта по модели 5W

Ответы на ключевые вопросы модели 5W приведены в таблице 2.1.

Таблица 2.1 – Обоснование проекта по модели 5W

Элемент	Ключевые вопросы	Ответ
WHAT?	Что делаем? Какую проблему решаем?	NotaCode — веб-IDE (PWA) класса «diagram as code»: диаграмма описывается кодом на собственном DSL, код проверяется по правилам выбранной нотации и отображается на холсте; диаграммы хранятся, версионизируются и экспортируются, генерация и исправление кода выполняются AI-ассистентом. Решаемая проблема: формальные нотации (IDEF0, IDEF1X, IDEF3, DFD, ERD, UML) строятся вручную в графических редакторах, а текстовые инструменты (PlantUML, Mermaid, D2) не проверяют правила методологических нотаций.
WHO?	Для кого? Кто заказчик?	Пользователи: студенты технических специальностей, системные и бизнес-аналитики, архитекторы ПО. Заказчик — автор проекта (Product Owner); результат принимается руководителем курсовой работы по дисциплине «Интернет-технологии и веб-программирование» (ч. 2) и преподавателем дисциплины «Управление проектами».
WHY?	Почему это важно? Почему сейчас?	Инструмент с единым DSL для инженерных и методологических нотаций, валидацией по стандартам и git-подобной историей версий на рынке отсутствует. Срок определяется учебным планом: курсовая работа и лабораторные работы ч. 2 выполняются в осеннем семестре 2026 г. на компонентах продукта.
WHEN?	Когда завершаем? (MVP и релиз)	Старт — 21.09.2026. MVP — 13.12.2026 (спринт S11). Защита курсовой работы — 18.12.2026 (плановая дата, подлежит уточнению). Версии Post-MVP (BPMN, сети Петри, произвольные диаграммы) — после защиты.
WHERE?	Где используется?	Браузеры Google Chrome, Microsoft Edge, Mozilla Firefox актуальных версий на настольных ПК; установка как PWA с офлайн-черновиками. Сервисы — облачный хостинг (Netlify, Render/Fly.io, Neon).

2.4 Бизнес-окружение проекта

Проект выполняется одновременно как курсовая работа по дисциплине «Интернет-технологии и веб-программирование» (ч. 2), как объект лабораторных работ той же дисциплины и как продукт с перспективой коммерциализации (версия Pro). Рынок инструментов моделирования разделён на два класса: текстовые DSL для инженерных нотаций (PlantUML, Mermaid, D2, Structurizr) и графические редакторы (draw.io, Ramus, BPwin), в которых методологические нотации строятся вручную. Бесплатные LLM API (Gemini, Groq, OpenRouter, Mistral) и бесплатные тарифы облачных сервисов обеспечивают реализацию AI-функций и развёртывания без бюджета. Сроки

задаются расписанием семестра: MVP требуется к защите курсовой работы в декабре 2026 г.

Проблемы предметной области и способы их решения продуктом приведены в таблице 2.2.

Таблица 2.2 – Проблемы и решения

№	Проблема (как есть сейчас)	Решение (что сделает продукт)
1	Диаграммы IDEF0, IDEF1X, IDEF3, DFD строятся вручную в графических редакторах; соответствие стандарту не проверяется, правка большой диаграммы занимает много времени.	Собственный DSL и профили нотаций с правилами валидации: ошибки выводятся с номером строки и колонки, для типовых ошибок предлагается автоисправление.
2	Текстовые инструменты (PlantUML, Mermaid, D2) не поддерживают методологические нотации и не проверяют их правила; для разных нотаций применяются разные инструменты и синтаксисы.	Единый язык и каталог нотаций «нотация → тип → уровень»: UML, ERD, IDEF0, IDEF1X, IDEF3, DFD в MVP, BPMN и сети Петри в Post-MVP.
3	История изменений диаграмм не хранится построчно, сравнение и откат версий недоступны; написание кода с нуля требует знания синтаксиса.	Неизменяемые коммиты с SHA-256-блоками, diff и откат; шаблоны, вкладка документации и AI-ассистент (режимы «Написать код», «Исправить код», «Исправить ошибку», «Спросить по документации»).

2.5 SMART-цель проекта

Формулировка цели по каждому критерию SMART приведена в таблице 2.3.

Таблица 2.3 – SMART-цель проекта

Критерий	Содержание
S – что конкретно делаем	Разработать MVP NotaCode: Web IDE (React, Monaco, SVG-холст), Edge Gateway (Node.js, Express), Language Service (Python, Lark) с нотациями волны 1 (UML, ERD, IDEF0, IDEF1X, IDEF3, DFD), сохранением на устройство и в БД, экспортом SVG и PNG и AI-ассистентом в режиме заглушки.
M – как измерим успех	Шаблон каждой из 6 нотаций волны 1 проходит Run без ошибок; время анализа — не более 300 мс для диаграммы из 200 узлов; покрытие тестами — 100 % (строки, ветви, функции); Lighthouse — не менее 90 во всех категориях; сквозной сценарий «код → Run → сохранение → экспорт» выполняется без ошибок.
A – почему достижимо	Требования и архитектура (C4, ADR) подготовлены в спринте S0; план содержит 13 недельных спринтов; сервисы подключаются поэтапно (walking skeleton → Language → Workspace); применяются только бесплатные сервисы.
R – как решает проблемы	Устраняет проблемы 1–3 раздела 1.2 и одновременно обеспечивает материал для курсовой работы и лабораторных работ ч. 2 (Gateway реализует требования ЛР1–ЛР8).
T – к какому сроку	MVP — до 13.12.2026; защита курсовой работы — 18.12.2026.

Итоговая SMART-цель: до 13.12.2026 разработать и продемонстрировать MVP веб-IDE NotaCode с валидацией и рендером 6 нотаций волны 1, сохранением, экспортом SVG/PNG и AI-заглушкой при покрытии тестами 100 % и оценке Lighthouse не ниже 90.

2.6 Правила приёмки и допущения

Заказчик принимает работу при выполнении следующих критериев:

1 Для каждой нотации волны 1 (UML, ERD, IDEF0, IDEF1X, IDEF3, DFD) код встроенного шаблона проходит Run без ошибок, диаграмма отображается на холсте.

2 Ошибки DSL выводятся в панели Problems, маркерами редактора и тостами Error/Warning/OK с указанием строки и колонки.

3 Сохранение на устройство и в БД, экспорт в SVG и PNG выполняются без ошибок; кросс-подсветка «элемент ↔ строки кода» работает в обе стороны.

4 Конвейер CI проходит: покрытие 100 % (строки, ветви, функции), линтеры без замечаний, в исходном коде отсутствуют комментарии.

5 Оценка Lighthouse — не менее 90 во всех категориях; пояснительная записка (20–30 с.) принята руководителем курсовой работы.

Допущения проекта и их влияние при невыполнении приведены в таблице 2.4.

Таблица 2.4 – Допущения проекта

№	Допущение	Влияние на проект (если не сбудется)
1	Преподаватель ИТиВП согласует применение Python-сервисов (FastAPI) наряду с рекомендованным стеком Node.js/Express.	Перенос Language и Workspace Service на Node.js, сдвиг MVP на 2–3 спринта.
2	Бесплатные тарифы Netlify, Render/Fly.io, Neon, CloudAMQP и Upstash сохраняются до даты защиты.	Смена хостинга и перенастройка развёртывания, риск недоступности демо-стенда на защите.
3	Дата защиты курсовой работы назначается не ранее 18.12.2026.	Сокращение объёма MVP: перенос требований Should have в Post-MVP.
4	Бесплатные ключи LLM API (Gemini, Groq, OpenRouter) остаются доступными.	AI-ассистент демонстрируется только в режиме заглушки.
5	Каждая лабораторная работа ч. 2 укладывается в один недельный спринт.	Вытеснение продуктивных задач лабораторными, сдвиг сроков MVP.

2.7 Содержание и границы проекта

Приоритизация требований по методу MoSCoW приведена в таблице 2.5.

Таблица 2.5 – MoSCoW-приоритизация требований

Приоритет	Требования
Must have	1. Редактор DSL (Monaco): подсветка синтаксиса, подсказки. 2. Run: разбор, валидация и рендер только по кнопке. 3. SVG-холст: сетка, масштаб, панорамирование. 4. Кросс-подсветка «элемент ↔ строки кода». 5. Нотации волны 1: UML, ERD, IDEF0, IDEF1X, IDEF3, DFD. 6. Валидация, панели Problems и Terminal, тосты Error/Warning/OK. 7. Сохранение (устройство, БД, GitHub, Google Drive), экспорт SVG, PNG, XML. 8. Шаблоны, вкладка документации, AI-ассистент (заглушка, 4 режима), светлая и тёмная темы.
Should have	1. Регистрация и вход (JWT, refresh-токены). 2. История версий, diff двух версий, откат. 3. Undo/redo (Ctrl+Z, Ctrl+Y). 4. Автоисправление (quick fix). 5. Импорт файлов DSL и проекта. 6. Интерфейс на русском и английском языках.
Could have	1. Авторендер после изменения кода. 2. Автосохранение в последнюю цель и формат. 3. Нотации волн 2–3 (BPMN, сети Петри). 4. Альтернативные движки (Kroki, PlantUML, Mermaid, Graphviz, D2). 5. Офлайн-режим PWA.
Won't have	1. Генерация диаграммы по изображению. 2. Редактирование на холсте с записью в код. 3. Совместное редактирование в реальном времени. 4. Одновременная работа с несколькими проектами.

Долю обязательных требований D_m , %, определяем по формуле

$$D_m = N_m / N \cdot 100, \quad (2.1)$$

где N_m – число требований Must have; N – общее число требований. $D_m = 8 / 23 \cdot 100 = 34,8 \%$, что не превышает 60 %.

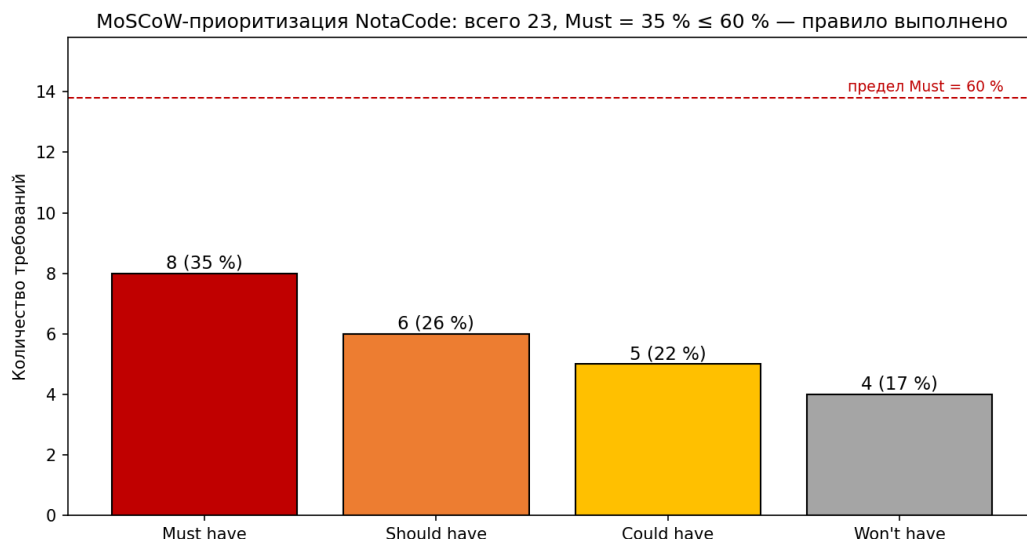


Рисунок 2.1 – Распределение требований по приоритетам MoSCoW

В проект не входят:

- мобильные приложения для iOS и Android;
- совместное редактирование диаграмм несколькими пользователями;
- платные LLM-модели, дообучение моделей, генерация диаграмм по изображению;
- биллинг и администрирование версии Pro;
- промышленная эксплуатация в Kubernetes и SLA.

2.8 Ограничения проекта

Ограничения проекта по категориям приведены в таблице 2.6; правовые ограничения определяются, в частности, Законом Республики Беларусь «О защите персональных данных» [5].

Таблица 2.6 – Ограничения проекта

Категория	Ограничения
Технические	TypeScript, React 19, Vite (Web IDE); Node.js 22, Express, Sequelize (Gateway); Python 3.13, FastAPI, Lark (сервисы предметной логики); PostgreSQL 17, MongoDB 7, Redis, RabbitMQ. Браузеры Chrome, Edge, Firefox актуальных версий. Run требует подключения к сети; офлайн доступны редактирование и черновики. Лимит размера DSL — 256 КБ.
Правовые	Закон Республики Беларусь № 99-3 «О защите персональных данных» (e-mail, пароль хранится в виде хэша argon2id). Лицензии зависимостей: MIT, Apache-2.0, EPL-2.0 (elkjs). Условия бесплатных тарифов LLM API. Ключи API хранятся только на сервере. OWASP ASVS L2.
Организационные	Один исполнитель в ролях Product Owner, разработчика и тестировщика; ревью выполняется AI-агентами. Процесс Scrumban: спринт — 1 неделя, WIP ≤ 2 . Стек ч. 2 задан методическими указаниями; применение Python-сервисов согласуется с преподавателем. Ветки лабораторных работ именуются по методичке.
Временные	Старт — 21.09.2026; спринт S0 (документация, фундамент) — до 04.10.2026; MVP — 13.12.2026; защита — 18.12.2026 (плановая дата); 13 недельных спринтов.
Бюджетные	Бюджет — 0 BYN: бесплатные тарифы Netlify, Render/Fly.io, Neon, CloudAMQP, Upstash и бесплатные ключи LLM API. Не включены: платные API, домен, платный хостинг. Резерв — буферный спринт S12.
Пользовательские	Пользователи владеют основами нотаций моделирования. Синтаксис DSL осваивается по встроенным шаблонам и вкладке документации; отдельное обучение не требуется.

2.9 Декомпозиция цели проекта

SMART-цель декомпозирована на 6 этапов и 22 задач (рекомендуемый диапазон шаблона — от 15 до 25 задач). Иерархический список работ приведён в таблице 2.7, схема — на рисунке 2.2.

Таблица 2.7 – Декомпозиция цели проекта

Код	Этап	Задачи
1.0	Анализ и проектирование	1.1 Формализовать требования (SRS: FR, NFR, UC) 1.2 Составить каталог нотаций волны 1 1.3 Спроектировать архитектуру (C4, ADR) 1.4 Подготовить мудборды интерфейса (VS Code, draw.io)
2.0	Фундамент разработки	2.1 Настроить монорепозиторий и CI (линтеры, покрытие 100 %) 2.2 Собрать Docker Compose (PostgreSQL, Redis, RabbitMQ) 2.3 Реализовать walking skeleton web → gateway → language
3.0	Серверная часть	3.1 Реализовать CRUD проектов и файлов в Gateway (Express) 3.2 Реализовать аутентификацию (JWT, refresh-токены) 3.3 Разработать грамматику DSL и парсер Lark с диагностикой 3.4 Реализовать профили и валидацию нотаций волны 1 3.5 Реализовать коммиты и историю версий (Workspace)
4.0	Клиентская часть	4.1 Сверстать оболочку IDE: activity bar, панели, темы 4.2 Интегрировать Monaco: подсветка, маркеры, подсказки 4.3 Реализовать SVG-холст (elkjs, pan/zoom, кросс-подсветка) 4.4 Реализовать сохранение и экспорт SVG/PNG
5.0	Интеграции	5.1 Реализовать AI-заглушку и адаптер LLM-провайдера 5.2 Реализовать 4 режима AI-ассистента 5.3 Подключить сохранение в GitHub и Google Drive
6.0	Тестирование и сдача	6.1 Выполнить e2e-тестирование основных сценариев 6.2 Провести аудит Lighthouse и безопасности 6.3 Оформить пояснительную записку и защитить курсовую работу

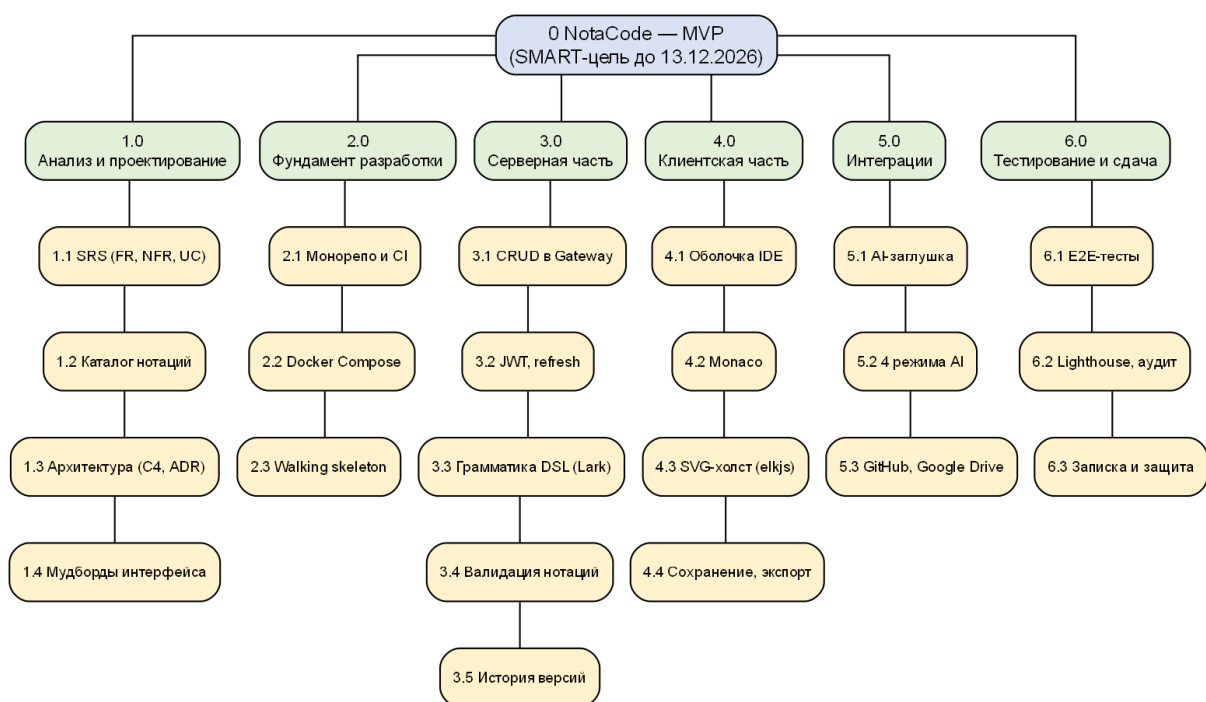


Рисунок 2.2 – Иерархическая структура работ проекта NotaCode

2.10 Анализ заинтересованных сторон

Заинтересованные стороны, их тип и значимость для проекта приведены в таблице 2.8. Влияние и интерес оценены по шкале от 1 до 5.

Таблица 2.8 – Идентификация заинтересованных сторон

№	Стейкхолдер	Тип	Почему важен
1	Автор проекта (Product Owner, разработчик)	внутренний	Формирует требования, реализует и принимает продукт. Влияние — 5, интерес — 5.
2	Руководитель курсовой работы (преподаватель ИТиВП)	внешний	Утверждает тему и стек, принимает пояснительную записку и защиту. Влияние — 5, интерес — 3.
3	Преподаватель дисциплины «Управление проектами»	внешний	Оценивает проектную документацию (концепция, WBS, риски). Влияние — 4, интерес — 2.
4	Конечные пользователи (студенты, аналитики, архитекторы)	внешний	Определяют требования к нотациям и удобству интерфейса. Влияние — 2, интерес — 5.
5	AI-агенты разработки (ревью, тесты)	внутренний	Выполняют ревью кода и контроль покрытия. Влияние — 3, интерес — 1.
6	Поставщики сервисов (LLM API, хостинг, GitHub, Google)	внешний	Задают лимиты и условия бесплатных тарифов. Влияние — 3, интерес — 1.

Стратегия работы определяется квадрантом матрицы: А – активное вовлечение (влияние 4–5, интерес 3–5); В – консультирование (влияние 1–3,

интерес 3–5); С – минимальный контроль (влияние 1–3, интерес 1–2); D – минимальное информирование (влияние 4–5, интерес 1–2). Распределение приведено в таблице 2.9 и на рисунке 2.3.

Таблица 2.9 – Распределение стейкхолдеров по квадрантам

Квадрант	Стратегия	Стейкхолдеры
A	Активное вовлечение	Автор проекта (5/5); руководитель курсовой работы (5/3).
B	Консультирование	Конечные пользователи (2/5).
C	Минимальный контроль	AI-агенты разработки (3/1); поставщики сервисов (3/1).
D	Минимальное информирование	Преподаватель дисциплины «Управление проектами» (4/2).

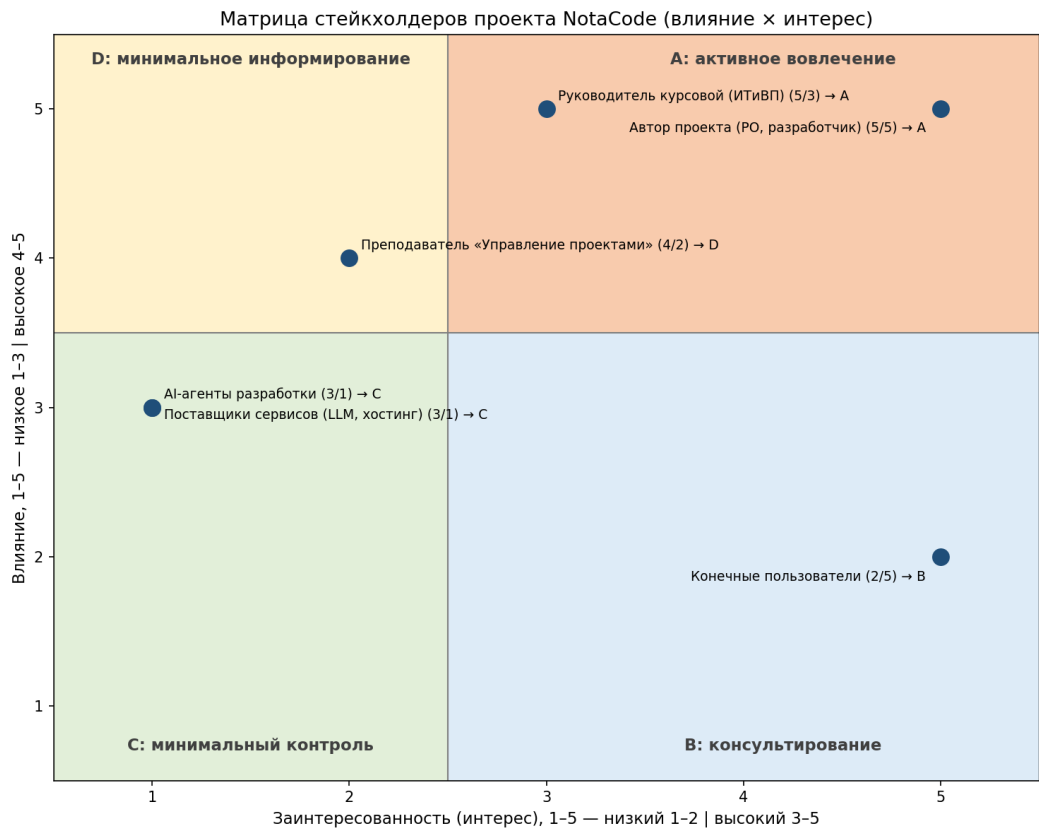


Рисунок 2.3 – Матрица заинтересованных сторон проекта NotaCode

ВЫВОДЫ

Проект обоснован, так как решает следующие проблемы: ручное построение формальных диаграмм без проверки стандарта, отсутствие единого текстового языка для инженерных и методологических нотаций, отсутствие построчной истории версий диаграмм.

Обязательный функционал (Must have) определён — 8 требований из 23 (35 %), границы проекта установлены перечнем исключений.

Ключевой стейкхолдер — заказчик (автор проекта, Product Owner); с руководителем курсовой работы выстроено активное взаимодействие по результатам спринтов.

Проект реализуем в рамках установленных ограничений: нулевой бюджет, один исполнитель, 13 недельных спринтов до защиты.

Разработанная концепция является исходными данными для построения иерархической структуры работ и календарного плана (практическое занятие № 2) и для анализа рисков проекта (практическое занятие № 3).

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Кашникова, И. В. Управление проектами : лекционные материалы и методические указания к практическим занятиям / И. В. Кашникова. – Минск : ИИТ БГУИР, 2026.

[2] Doran, G. T. There's a S.M.A.R.T. way to write management's goals and objectives / G. T. Doran // Management Review. – 1981. – Vol. 70, № 11. – P. 35–36.

[3] Руководство к своду знаний по управлению проектами (Руководство PMBOK®). – 6-е изд. – Newtown Square : Project Management Institute, 2017.

[4] NotaCode : проектная документация [Электронный ресурс]. – Режим доступа : <https://github.com/KsushaKhadzhinova/NotaCode>.

[5] О защите персональных данных [Электронный ресурс] : Закон Респ. Беларусь от 7 мая 2021 г. № 99-З. – Режим доступа : <https://pravo.by>.